

Table of contents

A Practical C Program : Program Information.....	3
The C Standard Library: Two Halves (Header Files, Precompiled Machine Codes).....	4
Static and Dynamic Linking.....	4
Inspecting Executables with “readelf”	5
Tracing Dependencies with ldd.....	6
Knowledge Check.....	7
GitHub Link.....	7

List of figures

Figure 1: C Program for procinfo.c.....	3
Figure 2: Compiling and Executing procinfo.....	3
Figure 3: Verifying the Locations.....	4
Figure 4: Creating Static and Dynamic Files.....	4
Figure 5: Executing Static and Dynamic Files.....	4
Figure 6: The ELF Header (readelf -h).....	5
Figure 7: The ELF Header (readelf -l).....	5
Figure 8: The ELF Header (readelf -d).....	6
Figure 9: The Use of ldd Command on Static and Dynamic Files.....	6

A Practical C Program : Program Information

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    time_t current_time = time(NULL);
    struct tm *time_info =
    localtime(&current_time);

    printf("Process Information\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path:/proc/self/exe\n");

    return 0;
}
```

Figure 1: C Program for procinfo.c

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ nano procinfo.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ gcc procinfo.c -o procinfo
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ./procinfo
Process Information
My Process ID (PID): 2920
Parent Process ID: 2793
Current Time: Sun Oct  4 16:57:04 2026
Executable Path:/proc/self/exe
```

Figure 2: Compiling and Executing procinfo

Summary

This program displayed basic information about a running process, including its PID and PPID. We compiled procinfo.c using GCC and executed the generated file.

The C Standard Library: Two Halves (Header Files, Precompiled Machine Codes)

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ uname -m
aarch64
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ ls /usr/include/stdio.h
/usr/include/stdio.h
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ ls /usr/include/unistd.h
/usr/include/unistd.h
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ ls /lib/aarch64-linux-gnu/libc.so*
/lib/aarch64-linux-gnu/libc.so  /lib/aarch64-linux-gnu/libc.so.6
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ ls /usr/lib/aarch64-linux-gnu/libc.a
/usr/lib/aarch64-linux-gnu/libc.a
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN. */
extern int fprintf (FILE *__restrict __stream,
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2$ grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
```

Figure 3: Verifying the Locations

Summary

We checked the locations of the C Standard Library files and observed both header and library files. Header files provide function declarations, while the library files contain the compiled code. This helped us understand how standard library functions are provided to C programs.

Static and Dynamic Linking

```
-rwxrwxr-x 1 shriya-awale shriya-awale 70K Oct  4 16:56 procinfo
-rw-rw-r-- 1 shriya-awale shriya-awale 470 Oct  4 16:56 procinfo.c
-rw-rw-r-- 1 shriya-awale shriya-awale 2.7K Oct  4 16:56 procinfo.o
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ gcc -static procinfo.c -o procinfo_static
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ls -lh procinfo_static
-rwxrwxr-x 1 shriya-awale shriya-awale 769K Oct  4 18:19 procinfo_static
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ gcc procinfo.c -o procinfo_dynamic
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ls -lh procinfo_dynamic
-rwxrwxr-x 1 shriya-awale shriya-awale 70K Oct  4 18:20 procinfo_dynamic
```

Figure 4: Creating Static and Dynamic Files

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ./procinfo_dynamic
Process Information
My Process ID (PID): 5079
Parent Process ID: 2793
Current Time: Sun Oct  4 18:21:20 2026
Executable Path:/proc/self/exe
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ./procinfo_static
Process Information
My Process ID (PID): 5081
Parent Process ID: 2793
Current Time: Sun Oct  4 18:21:30 2026
Executable Path:/proc/self/exe
```

Figure 5: Executing Static and Dynamic Files

Summary

We created both static and dynamic versions of the program and compared their execution and file sizes. We observed that the static file contains the required library code, while the dynamic file depends on shared libraries. This helped us understand the difference between the two linking methods.

Inspecting Executables with “readelf”

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:             ELF64
  Data:              2's complement, little endian
  Version:           1 (current)
  OS/ABI:            UNIX - System V
  ABI Version:       0
  Type:              DYN (Position-Independent Executable file)
  Machine:           AAarch64
  Version:           0x1
  Entry point address: 0x900
  Start of program headers: 64 (bytes into file)
  Start of section headers: 68944 (bytes into file)
  Flags:             0x0
  Size of this header: 64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 10
  Size of section headers: 64 (bytes)
  Number of section headers: 29
  Section header string table index: 28
```

Figure 6: The ELF Header (readelf -h)

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ readelf -l procinfo_dynamic
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x900
There are 10 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz             MemSiz             Flags   Align
PHDR             0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000230 0x0000000000000230 R       0x8
INTERP           0x0000000000000294 0x0000000000000294 0x0000000000000294
                 0x000000000000001b 0x000000000000001b R       0x1
  [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
LOAD             0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000cb4 0x0000000000000cb4 R E     0x10000
LOAD             0x000000000000fd40 0x0000000000001fd40 0x0000000000001fd40
                 0x0000000000002d0 0x0000000000002d8 RW      0x10000
DYNAMIC          0x000000000000fd50 0x0000000000001fd50 0x0000000000001fd50
                 0x000000000000200 0x000000000000200 RW       0x8
```

Figure 7: The ELF Header (readelf -l)

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ readelf -d procinfo_dynamic

Dynamic section at offset 0xfd50 contains 28 entries:
  Tag               Type              Name/Value
0x0000000000000001 (NEEDED)          Shared library: [libc.so.6]
0x0000000000000001 (NEEDED)          Shared library: [ld-linux-aarch64.so.1]
0x000000000000000c (INIT)            0x7e0
0x000000000000000d (FINI)            0xaf8
0x0000000000000019 (INIT_ARRAY)      0x1fd40
0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
0x000000000000001a (FINI_ARRAY)      0x1fd48
0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
0x0000000000000065 (GNU_HASH)         0x2b0
0x0000000000000005 (STRTAB)          0x480
0x0000000000000006 (SYMTAB)          0x2d0
```

Figure 8: The ELF Header (readelf -d)

Summary

We used different readelf options to inspect the internal information of our executable files. We observed the ELF header, program segments, and dynamic linking information. This helped us understand the structure and details stored inside an executable.

Tracing Dependencies with ldd

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x0000f8fbdca0c000)
libc.so.6 => /usr/lib/aarch64-linux-gnu/libc.so.6 (0x0000f8fbdca7b0000)
/lib/ld-linux-aarch64.so.1 (0x0000f8fbdca9d0000)
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week2/lab2.1$ ldd procinfo_static
not a dynamic executable
```

Figure 9: The Use of ldd Command on Static and Dynamic Files

Summary

We used the ldd command to check the shared library dependencies of our executable files. We observed that the dynamically linked file depends on libraries such as libc, while the static file does not. This helped us understand how libraries are loaded when a program runs.

Knowledge Check

1) What is the fundamental difference between a C header file and a shared library?

A: Header file is declarations whereas shared library is actual code.

2) What is static and dynamic linking?

A: Static is library code inside executable whereas dynamic is library code loaded at runtime.

3) What is ELF Header? Write down the commands to check headers.

A: It is the executable's basic information. Commands:

readelf -h <filename>, readelf -l <filename>, readelf -d <filename>

4) What does bookmark mean? provide an example.

A: It is a saved location for quick access. Eg :Browser bookmark → saved webpage.

GitHub Link: <https://github.com/shriya07-debug/Operating-Systems-Security-and-Networks.git>